

1. 수집 파이프라인의 정상 작동 흐름 (orchestrator.py 포함)

QualiJournal 관리자 UI에서 키워드 뉴스 수집을 실행하면, 백엔드가 orchestrator.py를 통해 수집 → 자동 선정(승인) → 발행 단계를 순차적으로 수행합니다 ¹. 구체적인 흐름은 다음과 같습니다:

- **(1) 기사 수집:** `/api/flow/keyword` 호출 시 `--collect-keyword <키워드>` 명령으로 orchestrator.py가 실행되어 해당 키워드의 기사를 검색하고 수집합니다. 필요에 따라 외부 RSS를 사용할 수도 있으며, 이 경우 `--use-external-rss` 옵션이 함께 전달되어 외부 뉴스 소스를 크롤링합니다 ¹. 수집된 기사들은 JSON 형태로 임시 저장되며, 이 JSON에는 수집 날짜(`date`), 키워드(`keyword`), 기사 목록(`articles`) 등이 포함됩니다.
- **(2) 요약 생성:** 수집 단계에서 각 기사에 대한 요약문을 생성합니다. (예: 뉴스 원문을 요약해 `summary` 필드에 추가). 이 과정은 orchestrator.py 내부에서 자동으로 이루어지며, 결과 JSON의 각 기사 객체에 영어/한국어 요약 필드가 채워집니다. (예: `summary_en`, `summary_ko` 등).
- **(3) Top 20 자동 승인:** 수집이 완료되면, 시스템은 상위 20개 기사를 자동으로 승인 처리합니다. `/api/flow/keyword`는 내부적으로 `--approve-keyword-top 20` 명령을 실행하여 수집된 기사들의 점수를 계산하고 상위 20개에 `approved=True` 및 상태 `ready`를 설정합니다 ². 이 단계는 `tools/force_approve_top20.py`에 구현되어 있으며, 기사 목록을 점수 순으로 정렬한 뒤 상위 N개(기본 20개)에 대해 `approved` 플래그를 True로 바꾸는 로직을 수행합니다 ³ ⁴. 만약 수집된 기사가 하나도 없다면 이 단계에서 "[!] 기사 없음" 경고만 출력되고 넘어갑니다 ⁵.
- **(4) 선정본 동기화:** 승인 단계 후 `_sync_after_save()` 함수가 호출되어, 승인된 기사들만 발행용 파일로 동기화됩니다 ⁶. 이 함수는 `selected_keyword_articles.json`에서 `approved=True`인 기사들을 필터링하여 별도의 최종 선정 파일(`selected_articles.json`)에 저장합니다 ⁷ ⁸. 동기화 과정에서 승인된 기사들의 상태를 `ready`로 통일하고 날짜를 보강한 뒤 출력 JSON을 저장하며, 동기화 성공 시 "fallback merge ok" 등의 로그를 남깁니다 ⁹.
- **(5) 발행 및 아카이브:** 마지막으로 `--publish-keyword <키워드>` 명령을 통해 발행 단계를 진행합니다 ¹⁰. 이 단계에서는 최종 선정된 기사들을 이용해 HTML/Markdown 리포트와 최종 JSON이 생성됩니다. orchestrator.py는 발행 과정에서 `archive/` 디렉토리에 금일자 키워드별 아카이브 파일들을 저장하며 (예: `archive/2025-10-26_IPC-A-610.html` 및 `.json`), 이전에 같은 날짜로 발행된 파일이 있으면 `_rollover_archive_if_needed` 함수를 통해 백업해 둡니다 ¹¹. 발행이 완료되면 `/api/flow/keyword` 응답으로 각 단계의 실행 결과(`steps`)가 반환되고, UI에서는 이를 기반으로 수집된 기사 리스트, 승인 현황, 발행 리포트 등을 표시합니다.

2. 수집→선정→발행 중간 산출물(JSON)의 위치와 구조

키워드 뉴스 파이프라인 실행 중 생성되는 중간 산출물 JSON 파일들은 다음과 같습니다:

- **작업본** `selected_keyword_articles.json` - 위치: 컨테이너의 `/app/data/selected_keyword_articles.json` 경로 (또는 환경에 따라 `/app/selected_keyword_articles.json`) ¹². 구조: 수집된 모든 기사의 목록을 담은 JSON 객체. 필드에는 수집 일자 (`date`), 키워드 (`keyword`), 기사 배열 (`articles`) 등이 포함됩니다. 예시 구조: `json`
{

```

    "date": "2025-10-26",
    "keyword": "ipc-a-610",
    "articles": [
      {
        "title": "기사 제목",
        "url": "원문 URL",
        "source": "매체 이름",
        "summary": "요약 본문...",
        "approved": false,
        "state": "candidate",
        "...": "기타 메타데이터"
      },
      ... (최대 20개+ 기사)
    ]
  }
}

```

} 수집 직후에는 `approved` 필드는 모두 `false`이며 `state`는 기본 `"candidate"`로 설정됩니다. **Top20 승인 후**에는 상위 20개 항목의 `approved`가 `true`로 바뀌고 `state`가 `"ready"`로 변경됩니다 2. 이 파일은 작업용 선정 파일로, 관리자 UI의 “선정” 탭 및 `/api/selection` 엔드포인트가 참조하는 데이터입니다.

- **최종본** `selected_articles.json` - 위치: 컨테이너 루트 (`/app/selected_articles.json`) 또는 `/app/data/selected_articles.json` 경로 12. 구조: 승인된 (최종 선정된) 기사들만 담고 있는 JSON 객체. `selected_keyword_articles.json`에서 `approved=True`인 기사들만 추려 동일한 구조로 저장한 것입니다. 즉, `date`와 `articles` 필드로 구성되며 `articles` 배열에는 승인된 기사만 포함됩니다 7 13. 예컨대 Top20 자동승인 후 발행되었다면 최대 20개의 기사만 이 파일에 남습니다. 이 파일은 발행용 선정 파일로, `/api/report`나 `/api/export` 등을 통해 리포트 생성 시 활용됩니다.

이 외에도 **아카이브 산출물**로서 `archive/` 디렉토리에 날짜+키워드 조합의 파일들이 생성됩니다. 예를 들어 `2025-10-26_IPC-A-610.json` 및 `.md`, `.html` 파일이 아카이브되어 저장됩니다. 이들 아카이브 JSON은 `selected_articles.json`과 동일한 내용이나, 히스토리 보관을 위해 날짜별로 분리된 파일입니다. 또한 Markdown 리포트(`*.md`)에는 기사 제목, 원문 URL, 요약, 편집자 코멘트 등이 포함되어 사람이 읽기 쉬운 형식으로 출력됩니다 14 15.

요약하면, **수집 단계**에서는 전체 기사 리스트가 작업본 JSON으로 저장되고, **승인/발행 단계**에서는 승인된 기사들만 최종본 JSON으로 추려지며, **발행 완료 후** 아카이브 디렉토리에 당일 키워드의 최종본이 누적되는 구조입니다.

3. `selected_keyword_articles.json`이 비는 4가지 가능한 원인 🔍

키워드 수집 후 선정 파일이 비어있는 현상이 발생하는 근본 원인으로 다음과 같은 네 가지를 고려할 수 있습니다:

1. **오케스트레이터 미실행 또는 실패:** `orchestrator.py` 스크립트 자체가 실행되지 않았거나 즉시 오류가 발생하여 **수집 단계가 진행되지 않은 경우**입니다. (예: 컨테이너 이미지에 `orchestrator.py`가 포함되지 않아 **스크립트를 찾지 못함**, 또는 스크립트 실행 직후 예외 발생 등).
2. **수집 대상 데이터 부재:** `orchestrator`는 정상 동작했지만 **키워드에 해당하는 기사가 원천에 없어서 결과 목록이 빈 경우**입니다. 즉, 검색/크롤링을 했으나 “ipc-a-610” 키워드로 relevant한 뉴스를 찾지 못해 수집 결과가 0건인 상황입니다. (예: 내부 DB나 기본 소스에 해당 키워드 기사가 없고, 외부 RSS 옵션도 사용되지 않음).

3. **파일 경로 또는 JSON 포맷 문제:** 수집된 결과가 있더라도 **선택 파일이 올바르게 기록되지 않거나 불완전하게 저장된** 경우입니다. 예를 들어, orchestrator가 결과를 임시 다른 경로에 저장하거나 JSON 작성 도중 에러로 파일이 손상되면 (`JSONDecodeError` 등), UI가 이를 읽지 못해 빈 것으로 인식할 수 있습니다¹⁶. 또한 환경 설정 차이로 파일 경로가 어긋난 경우 (e.g. 결과가 `/app/selected_keyword_articles.json`에 기록됐지만 UI는 `data/` 경로를 참조함) 파일이 비어있는 것처럼 보일 수 있습니다.

4. **파이프라인 단계 누락 또는 설정 오류:** 자동 승인이 제대로 이루어지지 않거나, 네트워크/설정 문제로 **특정 단계가 건너뛰어진** 경우입니다. 예를 들어, DB 모드나 API 키 설정 오류로 **수집 단계에서 데이터를 가져오지 못했는데도 오류 처리 없이 다음 단계로 넘어간** 경우, 혹은 Cloud Run 등 환경에서 **인터넷 접근이 막혀 크롤링에 실패한** 경우 등이 해당됩니다. 이때 orchestrator는 결과를 찾지 못해 빈 목록을 반환하거나, 승인 단계에서 처리할 기사가 없으므로 전체 파이프라인이 유효한 결과 없이 끝나게 됩니다.

4. 원인별 확인 가능한 로그/파일/API 응답 예시

각 원인에 대해, 실제 시스템에서 이를 식별할 수 있는 **징후나 로그 예시**는 다음과 같습니다:

- **원인 1 - 오케스트레이터 미실행/오류:** 서버 로그를 확인하면 `orchestrator.py` 실행에 실패한 흔적이 나타납니다. 예를 들어 **컨테이너에 스크립트가 없을 경우**, `/api/flow/keyword` 응답의 `steps` 배열에서 첫 번째 단계가 실패(`"ok": false`)하고 `stderr`에 아래와 같은 에러가 기록됩니다¹⁷:

```
"stderr": "orchestrator.py not found in image. (빌드 컨텍스트를 repo 루트로 잡았는지 확인)"
```

이는 **컨테이너 이미지 빌드 시 `orchestrator.py`가 포함되지 않은** 경우 발생하며, 서버 로그 (`server.log`)에도 해당 `stderr`가 경고로 남습니다¹⁸. 만약 `orchestrator.py`가 존재하지만 내부 코드 오류로 즉시 종료됐다면, 유사하게 `"ok": false`와 함께 `stderr`에 Python 예외 Traceback이나 에러 메시지가 포함되어 반환됩니다. 이러한 경우 `GET /api/logs/server.log` 또는 Cloud Run 로그에서 **"orch stderr:"** 키워드로 해당 오류 내용을 확인할 수 있습니다.

- **원인 2 - 수집 결과 없음:** orchestrator가 정상 실행되었지만 기사를 한 건도 찾지 못한 경우입니다. 이때 `/api/flow/keyword` 응답의 각 단계는 `"ok": true`로 표시될 가능성이 높으며, 오류는 없지만 결과 리스트가 비어 있게 됩니다. 예를 들어, 자동 승인 단계에서 `force_approve_top20` 스크립트는 기사 목록이 비었을 때 `"[!] 기사 없음"`이라는 메시지를 출력하고 종료합니다⁵. 해당 메시지는 `steps[1].stdout`에 남을 수 있으며, 서버 로그에도 INFO 레벨로 승인 결과 0건에 대한 기록이 나타날 수 있습니다 (예: `[OK] 0건 승인 완료 -> selected_keyword_articles.json`). 이 원인의 또 다른 증거는 **API 상태 조회**에서 드러납니다. `GET /api/status` 응답을 보면 `selection_total` 값이 0이고, `"selection_total": 0, "selection_approved": 0`으로 표시됩니다¹⁹. 또한 `POST /api/report`를 호출해보면 `"count": 0`와 함께 생성된 리포트에 `"(수집된 기사 없음)"`이라는 문구가 포함되어 반환됩니다^{20 21}. 요컨대 **오류 로그는 없지만**, 결과 관련 지표와 출력물에 기사 0건임이 드러나면 수집 대상 데이터가 없었던 상황으로 판단할 수 있습니다.

- **원인 3 - 파일 경로나 JSON 포맷 문제:** 이 경우는 로그보다는 **파일 자체를 검사**해야 원인을 알 수 있습니다. 서버에 접속 가능한 경우, 다음과 같은 명령으로 파일 상태를 확인할 수 있습니다:

```
$ ls -l /app/data/selected_keyword_articles.json /app/selected_keyword_articles.json
```

```
-rw-r--r-- 1 root root 0 Oct 26 10:00 /app/data/selected_keyword_articles.json
-rw-r--r-- 1 root root 25K Oct 26 09:59 /app/selected_keyword_articles.json
```

예시처럼 **data 경로의 파일 크기가 0이지만 루트 경로에 내용이 있다면**, orchestrator가 다른 경로에 결과를 기록한 상황입니다. 반대로 파일이 존재는 하나 크기가 0바이트이거나, `cat selected_keyword_articles.json` 했을 때 내용이 `{}` 또는 비정상적인 JSON이라면 **쓰기 중단/파일 손상**을 의심할 수 있습니다. 실제 `tools/repair_selection_files.py` 스크립트는 이런 상황을 대비해, JSON decode 오류 시 빈 배열로 치유하도록 만들어져 있습니다¹⁶. 그러므로 `selected_keyword_articles.json` 파일을 열었을 때 JSON 구조가 깨져 있거나 아예 비어 있다면, UI에서는 total 0으로 나타납니다. 이때 `/api/status`는 내부적으로 `_read_json` 호출 시 예외가 발생해 빈 dict `{}`를 받고, `selection_total`을 0으로 처리하므로 (`except Exception: return {}` 경로) 별도 오류 없이 0건으로만 보이게 됩니다²².

- **원인 4 - 단계 누락 또는 설정 오류:** 네트워크 차단이나 설정 오류로 수집이 진행되지 않은 경우, **로그에 미묘한 에러가 남거나 아무런 출력 없이 끝날** 수 있습니다. 예를 들어 Cloud Run 환경에서 인터넷 액세스가 불가했다면 외부 뉴스 API 호출이 타임아웃되어 orchestrator stderr에 해당 예외가 기록되었을 가능성이 큼 (ex: `requests.exceptions... connection timeout`). 이러한 오류는 서버 로그의 `"orch stderr:"` 항목으로 확인할 수 있습니다. 한편 DB 연결 등 설정 문제의 경우, orchestrator가 내부 DB 쿼리를 시도하다 실패하면 stderr에 접속 오류 메시지가 남았을 것입니다. 그러나 이러한 오류가 발생해도 코드상 적절히 처리되지 않으면 **steps 배열의 해당 단계가 실패로 표시되고** 이후 단계는 건너뛰어질 수 있습니다. 실제로 `/api/flow/keyword` 응답의 `"ok"` 값이 false이고 steps 중간에 `"ok": false`인 단계가 존재한다면, 그 단계에서 설정/환경 오류가 있었다고 추정할 수 있습니다²³. 또한 `selected_keyword_articles.json` 파일이 아예 생성되지 않았거나 (`ls`로 파일 부재 확인 가능) 생성되어도 `articles:[]`만 있는 경우, 원인2와 구별하기 위해 **로그의 오류 메시지 유무**를 반드시 함께 검토해야 합니다. 예를 들어, `"invalid or missing token"`과 같은 인증 오류나 API 키 관련 에러가 stderr에 찍혀 있다면 설정 문제가 원인임을 알 수 있습니다.

요약하면, **원인 4**는 다른 원인들과 증상이 겹칠 수 있지만, **로그에 남은 예외/에러 메시지와 API 응답의 실패 여부**가 식별 포인트입니다. 특히 Cloud Run 배포 환경에서는 권한/네트워크 설정 누락이 흔한 원인이므로, artifact registry 권한이나 외부 인터넷 접근 설정을 재확인해야 합니다 (이러한 설정 이슈는 별도 오류 없이 수집 결과만 0이 되게 할 수도 있음).

5. 문제 해결을 위한 빠른 점검 명령어

문제 원인을 진단하고 해결하기 위해 다음과 같은 **빠른 점검 명령어**를 활용할 수 있습니다 (관리자 토큰이 필요한 API는 `-H "X-Admin-Token:<TOKEN>"` 헤더를 포함하여 호출):

1. 파이프라인 재실행 및 상태 확인:

```
# 키워드 수집 파이프라인 재실행 (use_external_rss 옵션 포함 여부는 필요에 따라)
$ curl -X POST https://admin.standardai.co.kr/api/flow/keyword \
  -H "X-Admin-Token: <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"keyword":"ipc-a-610","use_external_rss":false}'
{"ok":false,"steps":[...] } # ok=false 또는 steps 내 stderr 확인

# 현재 선정 상태 확인 (요청 시 토큰 필요)
$ curl -H "X-Admin-Token: <TOKEN>" https://admin.standardai.co.kr/api/status
{"selection_total":0,"selection_approved":0,"keyword_total":0,...}
```

위처럼 `/api/flow/keyword` 를 다시 호출해보고 응답의 `steps` 를 확인합니다. `ok: false` 나 `steps` 배열 내 `"stderr"` 필드가 있다면 오류 내용을 분석하세요. 이어서 `/api/status` 로 `selection_total` 등이 0으로 나오는지 재확인합니다.

2. 로그 파일 점검:

```
# 서버 로그 목록 조회
$ curl -H "X-Admin-Token: <TOKEN>" https://admin.standardai.co.kr/api/logs
{"items":[{"name":"server.log","size":12345,...}, ...]}

# Orchestrator 관련 로그 내용 확인 (마지막 200줄)
$ curl -H "X-Admin-Token: <TOKEN>" https://admin.standardai.co.kr/api/logs/
server.log
2025-10-26T13:00:00 [INFO] run orch: orchestrator.py --collect-keyword ipc-
a-610 rc=0
2025-10-26T13:00:01 [WARNING] orch stderr: orchestrator.py not found in image.
(빌드 컨텍스트를 repo 루트로 잡았는지 확인)
...
```

`/api/logs` 엔드포인트로 서버 로그 파일 목록을 받고, `/api/logs/server.log` 등을 열어 최근 로그를 검토합니다. 여기서 orchestrator 실행 여부 (`run orch: ... rc=<code>`)와 any stderr 경고를 찾습니다. 위 예시는 orchestrator.py를 찾지 못한 경우로, 이런 로그가 있으면 원인1에 대응합니다. 또한 "기사 없음" 등의 출력이나 크롤링/API 오류 메시지가 있는지도 검색해봅니다.

3. 선정 파일 직접 확인: (Cloud Run에서는 곧바로 쉘 접속이 어렵지만, 필요 시 디버그용으로)

```
# 파일 존재 및 크기 확인
$ curl -H "X-Admin-Token: <TOKEN>" https://admin.standardai.co.kr/api/export/
md?preview=true
# (또는 Cloud Console 쉘 활용)
$ ls -lh /app/data/selected_keyword_articles.json /app/
selected_keyword_articles.json

# 파일 내용 출력 (작은 파일인 경우)
$ cat /app/data/selected_keyword_articles.json | jq . #jq로 포매팅 출력 (있다면)
```

`ls` 명령으로 `selected_keyword_articles.json`의 위치별 존재 여부와 크기를 확인합니다. `cat` 으로 내용을 열어 `articles` 배열이 비어있는지 (`"articles": []`) 확인하고, `date` 나 `keyword` 필드만 덩그러니 있는지 점검합니다. 만약 JSON 구문 오류로 열리지 않는다면 (`jq: error: Could not parse JSON`), 파일 손상이 의심되므로 `tools/repair_selection_files.py` 스크립트를 실행하는 것도 고려합니다. Cloud Run 환경에서는 직접 `ls` / `cat` 이 어렵다면, `/api/export/md?preview=true` 혹은 `/api/report` 요청을 통해 현재 선정 데이터가 반영된 리포트를 미리보기로 받아볼 수 있습니다. 기사 리스트가 빈 Markdown ("`(수집된 기사 없음)`") 표시)로 나온다면 파일에 기사가 없다는 증거입니다 ²⁴.

4. 구성 및 네트워크 체크:

5. **환경변수/시크릿 확인:** 배포 시 설정한 `QUALI_DB_MODE`, `ADMIN_TOKEN`, 외부 API 키 등이 올바르게 주입되었는지 확인합니다. 특히 DB 모드를 cloud로 했는데 DB 연결이 안 되면 local 모드로 재시도하거나, 외부 RSS 사용을 고려해야 합니다.
6. **외부 뉴스 소스 테스트:** `use_external_rss=true` 로 `/api/flow/keyword` 를 다시 실행해 보거나, `tools/force_approve_top20.py` 와 `tools/repair_selection_files.py` 를 수동으로 돌려 보는 방법도 있습니다. 이를 통해 외부 연결이 원활한지, 승인 로직은 동작하는지 빠르게 검증 가능합니다.
7. **아티팩트 권한 검증:** artifactregistry에 이미지 Pull 권한이 없어 최신 orchestrator가 반영되지 않았을 가능성도 있으므로, Cloud Run 런타임 서비스 계정에 Artifact Registry 읽기 권한이 있는지 점검합니다 (common fix 사항).

以上的 빠른 점검으로 원인을 좁힌 후, 아래 절차로 문제를 재현하고 해결까지 진행할 수 있습니다.

6. 재현 및 정상화까지의 절차 요약

마지막으로, 문제 상황을 재현하고 해결하는 일련의 과정을 단계별로 정리하면 다음과 같습니다:

1. **문제 재현:** 관리자 UI에서 문제의 키워드(`ipc-a-610`)에 대해 "수집 → 승인 Top20 → 발행" 순으로 작업을 수행합니다. 이후 `/api/status` 를 조회했을 때 `selection_total` 이 0으로 표시되고, `/api/selection` 이나 UI 선정 탭에 기사가 나타나지 않으며, `/api/report` 출력 내용이 비어 있음을 확인합니다 (재현 완료).
2. **원인 진단:** 위의 5번 점검 명령어들을 활용하여 시스템 상태를 면밀히 조사합니다.
3. `/api/flow/keyword` 응답의 `steps` 와 `server.log` 로그를 분석해 오케스트레이터 실행 오류 여부를 확인합니다. 오류 메시지 "`not found in image`" 가 있다면 배포 문제(원인1)로 판단하고, 아니면 다음을 진행합니다.
4. 오류가 없다면 수집 결과 부재(원인2)를 의심합니다. `/api/status` 의 count와 `/api/report` 미리보기에 "수집된 기사 없음" 문구가 있는지 확인합니다.
5. 수집 결과가 없음이 확실하면, 외부 RSS 미사용 또는 데이터 소스 부족이 원인이므로, 외부 뉴스 활성화를 고려합니다. 반면, 결과가 없는데도 뭔가 어색한 로그(예: 예외 stacktrace)가 있다면 설정/네트워크 오류(원인4)로 전환하여 API 키, VPC 설정 등을 점검합니다.
6. 한편, 파일이 존재하고도 읽히지 않는 정황이 있으면 파일 손상/경로 이슈(원인3)로 간주합니다. 이때 `repair_selection_files.py` 를 실행하거나, 올바른 경로로 파일이 저장되도록 코드/경로 설정을 교정해야 합니다.
7. **문제 해결:** 진단된 원인에 따라 해결책을 적용합니다.
8. 원인 1: CI/CD 배포 수정 - Docker 빌드 컨텍스트를 레포지토리 루트로 지정하고, `orchestrator.py` 를 이미지에 포함시킨 새 버전을 배포합니다. 배포 후 `/api/flow/keyword` 실행 시 더 이상 "not found" 오류가 없는지 확인합니다.
9. 원인 2: 외부 데이터 소스 활용 - UI나 API에서 `use_external_rss=true` 로 수집을 다시 시도하거나, `config.json` 에 해당 키워드와 연관된 RSS 피드를 등록합니다. 또한 키워드 철자나 검색어가 적절한지 재검토하여 다른 유의어/대체어로도 테스트해봅니다.
10. 원인 3: 데이터 파일 복구 및 경로 정리 - `tools/repair_selection_files.py` 스크립트를 실행하여 손상된 JSON을 정상화하고 승인/발행본을 재생성합니다²⁵. 또한 애플리케이션 설정에서 작업본과 발행본의 경로가 일관되게 `data/` 로 향하도록 수정하여, 앞으로 경로 불일치가 없도록 합니다. (예: `server_quali.py` 의 `SEL_WORK`, `SEL_PUB` 경로 재확인)

11. 원인 4: **환경 설정 보완** – Cloud Run의 경우 **VPC egress 설정**을 인터넷 허용으로 변경하거나 필요한 API에 대한 VPC 커넥터를 설정합니다. 또한 누락된 API 키나 잘못된 DB 접속 정보가 있다면 이를 올바르게 세팅하고 시크릿을 주입합니다. Artifact Registry 권한 문제였다면 서비스 계정에 `artifactregistry.reader` 권한을 추가하고 이미지를 재배포합니다.
12. **정상화 확인:** 수정 사항을 적용한 후 **동일한 키워드로 다시 파이프라인을 실행**합니다. 이제 `/api/status` 에서 `selection_total` 이 수집된 기사 수로 증가하고, `selection_approved` 도 20 (또는 실제 승인된 수)로 나타나야 합니다. `/api/selection` 조회 시 기사 리스트 배열이 채워져 나오고, `/api/report` 를 생성하면 Markdown에 기사 목록과 요약이 포함되어 있을 것입니다 ²⁰. UI 화면에서도 키워드에 대한 뉴스 목록과 편집장 코멘트 입력란 등이 정상 표시되는지 확인합니다. 마지막으로, **서버 로그에 더 이상 오류나 경고가 발생하지 않았는지** 모니터링하여 문제 재발 징후가 없는지 점검합니다. 모든 단계가 정상적으로 완료되었다면 선정 파일(empty) 문제는 해결되어, 키워드 뉴스 수집-발행 기능이 의도대로 동작하게 됩니다.

1 6 9 10 11 14 15 17 18 19 20 21 22 23 24 `server_quali.py`
https://github.com/cdmacs1003-cyber/quali-journal/blob/49a0e32cd820715979aba34e9e890c8db9c30729/server_quali.py

2 3 4 5 `force_approve_top20.py`
https://github.com/cdmacs1003-cyber/quali-journal/blob/49a0e32cd820715979aba34e9e890c8db9c30729/force_approve_top20.py

7 8 12 13 `sync_selected_for_publish.py`
https://github.com/cdmacs1003-cyber/quali-journal/blob/49a0e32cd820715979aba34e9e890c8db9c30729/sync_selected_for_publish.py

16 25 `repair_selection_files.py`
https://github.com/cdmacs1003-cyber/quali-journal/blob/49a0e32cd820715979aba34e9e890c8db9c30729/repair_selection_files.py